

智能閹雞訓練機 — 技術知識點備忘

Smart Chicken Trainer · Competition Reference

作者：JIPX · 比賽日：2026-04-29

【一頁速記】重要數字、規格、關鍵字

類別	項目	數值 / 規格
MCU	Arduino Mega 2560	ATmega2560 / 16MHz / 256KB Flash / 8KB SRAM / 4KB EEPROM
MCU	Wemos D1 Mini	ESP8266 / 80 MHz / 4MB Flash / 80KB heap / 802.11 b/g/n
顯示	MCUFRIEND TFT	480 × 320 / RGB565 / 16-bit color / SPI
音效	DFPlayer Mini	MH2024K-24SS / UART 9600 / MP3 / SD card
感測	霍爾感測器	A3144 / digital output / 3 顆 (頭/胸/腹)
通訊	UART baud	9600 (Mega↔D1 / Mega↔DFPlayer / Serial Monitor)
通訊	HTTPS	WiFiClientSecure + setInsecure() / 自動 wake-up 60s
影像	RAW 圖檔	480×320×2 = 307,200 bytes / frame (≈ 300 KB)
雲端	Render free tier	冷啟動 30~50 秒 / Singapore region
後端	Python	3.11.10 (因 cgi 模組於 3.13 被移除)
協議	JSON over HTTPS	POST /api/sync, GET /api/time

1. 硬體系統架構 (Hardware Architecture)

整體架構

採用「Mega + D1 Mini」雙 MCU 架構：Mega 為主控、D1 為 WiFi 橋接器。兩塊板子透過 UART (Serial1) 互通。

元件	型號 / 規格	在系統中的角色
主控 MCU	Arduino Mega 2560	感測讀取 / TFT 顯示 / 音效控制 / 主邏輯
WiFi 橋	Wemos D1 Mini (ESP8266)	連 WiFi → POST 雲端 → 把結果回給 Mega
顯示	MCUFRIEND TFT 3.5"	480×320 觸控彩屏，當 UI 跟即時回饋
音效	DFPlayer Mini	SD 卡 MP3 解碼 + 喇叭驅動
感測	霍爾感測器 ×3	偵測閹雞模型「頭/胸/腹」是否被觸碰
儲存	Micro SD 卡 (FAT32)	存 RAW 圖檔 + MP3 (DFPlayer 跟 TFT 各自讀)
電源	5V USB / DC	Mega 供電 → D1 從 5V 經穩壓拿 3.3V

Mega 腳位配置 (Pinout)

腳位	功能	連接到
Pin 16 (TX2) / Pin 17 (RX2)	Serial2 — 9600 baud	DFPlayer Mini RX/TX

腳位	功能	連接到
Pin 18 (TX1) / Pin 19 (RX1)	Serial1 — 9600 baud	D1 Mini RX/TX
Pin 11 / 12 / 13 (SoftSPI)	SD 卡 SPI	TFT 模組上的 SD 卡槽 (CS=10)
Pin 2 / 3 / 4	Digital input	霍爾感測器 ×3
A0~A3, 8, 9	TFT 控制	MCUFRIEND TFT (LCD_RD/WR/CS/CD/RST + 數據腳)

2. 通訊協議 (Communication Protocols)

協議	速度 / 規格	用在哪
UART (Serial)	9600 baud, 8N1	Mega↔D1, Mega↔DFPlayer, USB Debug
SPI	~8 MHz	TFT 顯示器資料傳輸 (16-bit parallel) / SD 卡 (SoftSPI)
WiFi 802.11 b/g/n	2.4 GHz, WPA2-PSK	D1 Mini 連手機熱點 (JIPX7)
DHCP	IPv4	D1 自動取得 IP (例: 172.20.10.x = iPhone hotspot)
TCP/IP + TLS 1.2	HTTPS port 443	D1 → Render 雲端伺服器
HTTP/1.1	POST + JSON body	/api/sync 上傳成績 / /api/time 校時

自訂應用層協議 (Mega ↔ D1 Mini)

為了減少 Mega 的工作量，雙方用人類可讀的「行協議」(line protocol)：

```
Mega → D1: SCORE:88,SURV:90,DUR:120,DIFF:2,NAME:Device\n
D1 → Mega: SYNC_OK:RANK:22:CODE:SKC23:PIN:5757\n
D1 → Mega (■■): WIFI:OK\n ■ WIFI:NO\n (■ 10 ■)\n
D1 → Mega (■■): TIME:2026-04-29 09:30\n
```

優點：除錯時直接打開 Serial Monitor 就看得懂；壞處：要自己處理跳脫。

HTTPS 為何用 setInsecure()

ESP8266 跑完整 TLS 1.2 + 憑證驗證 (X.509 + RSA-2048) 需要 ~30KB heap，但 D1 Mini heap 只有約 51KB，光 TLS 就吃掉 60%。所以採取「只加密、不驗證」策略：流量仍走 TLS 加密 (中間人看不到內容)，但跳過憑證鏈驗證。對成績上傳的場景已足夠 (DEVICE_KEY 在後端做認證)。

3. 韌體技術 (Firmware)

語言與框架

- Arduino C++ (g++ avr-gcc 7.3 / xtensa-lx106-gcc)
- Adafruit GFX + MCUFRIEND_kbv (TFT 驅動)
- U8g2 (中文字型, CJK glyph rendering)
- DFPlayer Mini Library
- ESP8266HTTPClient + WiFiClientSecure (D1 端)
- SoftSPI (SD 卡專用, 避開 MCUFRIEND 佔用 SPI 腳)

記憶體優化技巧 (AVR 嚴格 8KB SRAM 限制)

技巧	原理	效果
F() macro / PROGMEM	把字串常數放 Flash 而非 SRAM	節省幾百 bytes SRAM
不用 String 類別	避免 heap fragmentation	長時間執行不會崩
避免大型區域陣列	改成 static / 全域	Stack 不會爆
EEPROM 存設定	電源關了也保留	4KB 永久儲存
結構化 const 表	devLabels[] 一次性查找	減少 if-else 鏈

非阻塞設計 (millis() vs delay())

整個 loop() 用 millis() 做時間軸而非 delay(), 這樣音效播放、感測器掃描、TFT 重繪、WiFi 心跳可以「同時」進行。例如：

```
if ((uint32_t)(millis() - lastTick) >= 1000) {
    lastTick = millis();
    updateExamTimer();
}
```

狀態機 (State Machine)

用 enum Mode 管理 11 種模式：MENU / WELCOME / TEACH / PRACTICE / EXAM / EXAM_PICK / EXAM_POPUP / EXAM_UPLOAD / EMERGENCY / DEV / TEST。每個模式有自己的 show*()、handle*Touch(), 避免控制流爆炸。

4. 影像處理 (Display & Graphics)

為何用 RGB565 RAW 格式？

評估過三種選擇：

格式	解碼速度	檔案大小	是否採用
BMP 24-bit	需轉 RGB565 (慢)	$480 \times 320 \times 3 = 460\text{KB}$	太大
JPEG	AVR 解碼 ~5 秒/張	~30KB	AVR 太慢
PNG	AVR 沒像樣的 lib	~50KB	不可行
RGB565 RAW	直接 push 到 TFT	$480 \times 320 \times 2 = 307\text{KB}$	✓ 採用

RAW 流程

- 用 Photoshop / GIMP 製作 480×320 圖
- 透過 imagemagick 或自寫 Python script 轉成 little-endian RGB565
- 存到 SD 卡 (FAT32)
- Mega 每次顯示時 streaming 讀檔 → 寫到 TFT framebuffer
關鍵：streaming 讀！不能整張載入 SRAM ($307\text{KB} \gg 8\text{KB}$)，必須用 `SDFile.read()` 每次讀 ~256 bytes 邊讀邊推給 TFT。

中文字型 (U8g2)

MCUFRIEND 內建字型只有 ASCII。中文用 U8g2 的子集字型：`u8g2_font_wqy12_t_chinese3` (~574 字元，含蓋常用字)。Flash 大約佔 60KB。

5. 音效系統 (Audio)

DFPlayer Mini 工作原理

- 內建 MP3 硬體解碼晶片 (MH2024K-24SS) — Mega 不需自己解 MP3
- Mega 透過 UART 9600 送指令封包 (10-byte 格式)
- DFPlayer 從 SD 卡讀檔 → 解碼 → 直接驅動喇叭 (3W 內建放大器)

SD 卡資料夾結構

DFPlayer 規定資料夾名稱必須是兩位數字 (01~99)，檔名前綴三位數字 (001~999)：

資料夾	用途	範例
/01/	教學音效	001.mp3 (T1) ~ 009.mp3 (TEND)
/02/	練習回饋音	001=OK, 002=WARN, 003=NG
/03/	評分音效	001=扣分, 002=結束, 003=雞死
/04/	Welcome 介紹音	001=教學介紹, 002=練習介紹, 003=評分介紹

山寨晶片陷阱 (MH2024K)

這款 DFPlayer 用的是 MH2024K-24SS 晶片，與 DFRobot 原廠 (YX5200) 行為不同：

- 必須 stop() → delay(50ms) → play(), 否則指令會被吞掉
- 偶爾誤報 CardRemoved 事件 (其實 SD 卡好的)
- 解法：自己包一層 playTrack(folder, file), 內建 retry + 容錯

音檔規格

CBR 128 kbps / 44.1 kHz / Stereo / MP3 — 最穩相容組合。VBR 跟 22.05 kHz 在某些山寨 chip 會出現「播一半停住」。

6. 雲端與網路 (Cloud & Network)

WiFi 連線流程

1. D1 開機 → WiFi.begin(SSID, PASS)
2. 主動掃描 → 4-way handshake (WPA2-PSK)
3. DHCP discover → offer → request → ACK
4. 取得 IP (例：172.20.10.14 = iPhone hotspot 子網)
5. 內部 NAT → 出公網

REST API 設計

端點	方法	功能
/api/sync	POST	D1 上傳成績 → server 回 rank + rename_code/pin
/api/time	GET	校時 (Asia/Taipei), 給開發者選單 TIME CAL 用
/api/leaderboard	GET	排行榜資料 (網站 frontend 用)
/api/status	GET	健康檢查
/api/announcement	GET	系統公告

Render PaaS 部署

- Free tier — 15 分鐘無流量會冷凍 (cold start)
- 重新喚醒約 30~50 秒, 因此 Mega 跟 D1 timeout 都設 60 秒
- git push → 自動 build → deploy (CI/CD pipeline)
- Region: Singapore (對台灣延遲約 70ms)

可靠性策略

- WiFi 心跳：D1 每 10 秒回報狀態 → Mega 左上角圖示 (綠/紅/閃黑)
- Cold start 容錯：Mega timeout 60 秒, 給 server 喚醒空間
- 重連邏輯：D1 連續 60 秒沒連上 WiFi 自動重試
- 即時回饋：UPLOADING 動畫, 告訴使用者「我在做事」

7. Python 後端 (Backend)

技術選型理由

選擇	替代方案	為何不選
Python 3.11.10	3.13	3.13 移除 cgi 模組，本專案需要
http.server (stdlib)	Flask / FastAPI	少一層依賴，部署更簡單
JSON 檔案存資料	PostgreSQL / SQLite	比賽資料量小 (<100 筆)，純 file 即可
Asia/Taipei timezone	UTC	比賽現場顯示要在地化
datetime.now(TZ)	datetime.utcnow()	避免 timezone 混亂

資料模型 (records.json)

每筆紀錄包含：id (8 字元 UUID 子集) / name / score (0-100) / survival (%) / duration (秒) / difficulty (EASY/NORMAL/HARD) / date (Asia/Taipei) / approved / rename_code / rename_pin / device_id

匿名改名機制

玩家不需註冊帳號就能改自己的名字：

- 上傳成功時，server 隨機產生 5 字 code (e.g. SKC23) + 4 位 pin (e.g. 5757)
- Mega TFT 把 code/pin 顯示給玩家
- 玩家上網站 → 輸入 code+pin → 改名字 (僅自己這筆)
- 等同 capability-based access — 沒帳號系統的代表方案

8. 前端網站 (Frontend)

技術選型

- 純 HTML + CSS + Vanilla JavaScript — 不用框架 (React/Vue)
- 單檔 index.html — 快速載入、適合公網部署
- Chart.js (CDN) — 統計圖表
- 不用 Tailwind — 自寫 CSS variables, 深色/淺色一鍵切換

功能

- 排行榜 (按綜合分/分數/存活率/日期排序)
- 統計頁 (分數分布、難度比較、趨勢圖)
- 個人查詢 (用 device 或 name)
- 圖片倉 (上傳作品照片)
- 線上模擬器 (模擬完整流程體驗)
- 系統指南 (怎麼操作)

國際化 (i18n)

支援 4 種語言：繁體中文 / English / 日本語 / 한국어。用 LANG[lang][key] 字典 + data-i18n attribute。切換語言不重新整理頁面。

深色模式

CSS variables (--bg, --text, --accent...) + body.dark class。localStorage 記住偏好。符合 macOS / iOS / Windows 11 設計趨勢。

9. 評分演算法 (Scoring)

扣分制 (Deduction System)

起始 100 分，每次錯誤扣分：

難度	錯誤一次扣多少	評分時長
EASY (簡單)	- 10 分	60 秒
NORMAL (普通)	- 15 分	60 秒
HARD (困難)	- 25 分	60 秒

存活率 (Survival Rate)

= (倖存秒數 / 總時長) × 100%。中途死亡會大幅降低存活率。

綜合分 (Composite Score)

= score × 0.6 + survival × 0.4 — 平衡準確度與穩定度。排行榜預設按綜合分排序。

全球排名 (Global Rank)

跨難度合併排名 — 玩 HARD 的高分自然排前面。用 server-side sort 計算，不在 client 端做。

10. 評審可能會問 — Q&A; 速答

Q1：為什麼用 Arduino 不用 Raspberry Pi？

A：Arduino 即時性更好 (沒 OS context switch)、開機 1 秒、功耗 200mA、成本約 \$300。Pi 開機 30 秒、功耗 800mA、成本 \$1500，且要管 OS 升級。比賽現場「按下去要立刻有反應」— Arduino 完勝。

Q2：為什麼要兩塊板子，不能用一塊 ESP32 全包？

A：可以但有 trade-off。ESP32 雙核 + WiFi 看起來夠用，但：(1) 我們已有 Mega 跑得很穩 (2) Mega 的 4 個 hardware UART 對 DFPlayer 跟 D1 同時通訊很重要 (3) D1 Mini 處理 WiFi/HTTPS 時 CPU 滿載 — 分離出去 Mega 可以專心做即時控制 (4) 模組化好維護，WiFi 出問題不影響核心功能。

Q3：HTTPS 連線安全嗎？setInsecure() 不就是不安全？

A：誤解。setInsecure() 只是「不驗證憑證鏈」— 連線本身仍走 TLS 1.2 加密，中間人攔截看到的還是密文。憑證驗證需要 ~30KB heap，ESP8266 吃不下。我們用後端 DEVICE_KEY (CHICKEN_SECRET_2026) 在 application 層做認證，等同 API key — 沒 key 上傳會被 server reject。

Q4：Mega 只有 8KB SRAM，怎麼處理 480×320 全螢幕影像 (300KB)？

A：Streaming 讀取！每次只從 SD 卡讀 256 bytes 到 buffer，馬上推給 TFT framebuffer，再讀下一段。一張圖總共讀 1200 次，但每次只佔 256 bytes SRAM。代價是慢一點 (~700ms 一張)，可接受。

Q5：山寨 DFPlayer 晶片如何處理？

A：MH2024K-24SS 跟原廠 YX5200 雖然 pin 相容但 firmware 行為不同：(a) 必須 stop()→delay(50ms)→play() 否則指令會被吞 (b) 偶爾誤報 CardRemoved (其實 SD 沒拔)。對策：自己包 playTrack() 函式做容錯。另外音檔強制重新編碼為 CBR 128k / 44.1kHz / Stereo 才不會「播一半停」。

Q6：Render 雲端冷啟動 30~50 秒，玩家要等很久怎麼辦？

A：三層應對：(1) Mega timeout 設 60 秒給足時間 (2) 上傳頁顯示「first upload may take up to 60 sec (cloud wake-up)」管理期待 (3) UPLOADING ... 動態 dot 動畫，讓玩家知道系統在動。後續可加 keep-alive ping 進一步減少冷啟。

Q7：玩家可以改自己的名字嗎？怎麼防止亂改別人的？

A：用 capability-based 驗證：上傳時 server 隨機產生 5 字 code + 4 位 pin，Mega TFT 顯示給玩家。玩家上網站輸入 code+pin 才能改該筆紀錄。等同「你拿到鑰匙才能開鎖」— 不需要帳號系統。

Q8：如果手機熱點斷線，系統會崩嗎？

A：不會。D1 每 10 秒檢測 WiFi 狀態並回報 Mega。離線時 Mega 左上角圖示變紅，但 Mega 本機評分功能完全不受影響 — 只是無法上傳。WiFi 恢復後自動重連 (D1 已寫 reconnect 邏輯)。

Q9：未來怎麼擴展？

A：(1) 多裝置：REST API 已標準化，新裝置只要會 POST /api/sync 就能加入 (2) 多語言：i18n 結構已就位，加新語言只要新增 LANG['xx'] = {...} (3) 雲端 ML 分析玩家動作 (上傳影片) (4) Mega 升級為 ESP32 全包，去掉 D1 Mini 簡化線路。

Q10：開發過程遇到最難的 bug 是什麼？

A：近期：Server 用 Python json.dumps() 預設輸出 "key": "value" (冒號後有空格)，但 D1 firmware 一開始寫死找 "key": "value" (沒空格) → indexOf 永遠找不到 → rename_code/pin 永遠空。Debug 過程：用 logic analyser 攔截 D1↔Mega serial 流，發現 SYNC_OK:RANK:N:CODE::PIN: 是空的，但 server 回的 JSON 明明有值，回頭追字串匹配才發現。修法：改成「找 key → 跳到下個 quote → 讀到下個 quote」讓 parser 對空白格式不敏感。

Q11：成本大約多少？

A：Arduino Mega ~\$300, Wemos D1 ~\$80, MCUFRIEND TFT ~\$200, DFPlayer Mini ~\$60, 霍爾感測器 ×3 ~\$60, SD 卡 ~\$80, 電源/線材 ~\$100, 外殼/結構 ~\$200。電子料總計約 NT\$ 1,080，售價 NT\$ 1,499 (網站列出) 含結構件與組裝。

Q12：如果評審說「我看不到 WiFi 圖示」？

A：左上角圖示有三態：閃爍黑 = 偵測中 (D1 還沒回報) / 綠 = 已連 / 紅 = 未連。如果一直閃黑色，代表 D1 沒在跑 — 檢查 D1 是否上電、JIPX7 熱點是否開著。圖示顏色由 D1 每 10 秒推送的 WIFI:OK / WIFI:NO 心跳決定。

11. 開發工具與流程 (Toolchain)

工具	用途
arduino-cli	命令列編譯 / 上傳 — 比 Arduino IDE 快又可 script
Python 3 + pyserial	直接在 Mac 模擬 Mega 或 D1 (除錯用)
curl	直接打 REST API 測試後端
Git + GitHub	版控 / D1+server 自動部署 Render
Render	PaaS — push 即自動部署
VS Code / Claude Code	編輯器 + AI 配對程式設計

除錯方法

- Mega 端：USB 接 Mac → arduino-cli monitor 看 Serial.println 輸出
- D1 端：USB 接 Mac → 看 ESP8266 的 Serial.println 輸出
- 雙端互通：模擬一邊送資料給另一邊 (用 pyserial)
- 雲端：curl -v 看完整 HTTP 流量
- 前端：Chrome DevTools Network panel
- Live debugging：把問題的 Mega 接 USB，python 攔截 Serial1 流量看封包

CI/CD 流程

git push → GitHub → Render 觸發 webhook → 拉最新 code → pip install (build) → run server.py → 約 90 秒後上線。前端 (HTML/CSS/JS) 改完幾乎瞬間生效。

12. 資安考量 (Security)

風險	緩解措施
HTTPS 中間人	TLS 1.2 加密 — 雖然不驗證 cert，內容仍密文
假裝裝置上傳	DEVICE_KEY 認證 (CHICKEN_SECRET_2026) — 沒 key 拒絕
SQL Injection	無 — 沒用 SQL，純 JSON 檔案
XSS	前端 textContent 而非 innerHTML 設值
CSRF	寫入端點都需 device_key — 沒 cookie 機制
公網暴露	Render 免費版只開 80/443 — 沒額外開放
DEVICE_KEY 洩漏	韌體 binary 不公開 — 風險可控
名字內容過濾	server 限制長度 16 字元、過濾 control chars

13. 30 秒講解 (Elevator Pitch)

「這是用 Arduino Mega 當主控、Wemos D1 Mini 當 WiFi 橋接器的智能閹雞訓練機。Mega 透過三顆霍爾感測器讀取觸控位置，TFT 即時顯示分數、DFPlayer 播放對應音效。玩完後成績透過 D1 Mini 走 HTTPS 上傳到 Render 雲端的 Python server，玩家可以上網站看排行榜跟改名字。整套系統採模組化設計，Mega 跟 D1 之間用人類可讀的 line protocol 溝通，方便除錯跟未來擴展。」

【祝比賽順利！加油】